

Lesson 8 — Sparse Matrix Operator Overloading in C++

1. Where We Are in the Course

You have already learned the main sparse matrix ideas.

Lesson	Main idea
Lesson 1	A sparse matrix stores mostly zero values, so we store only non-zero values.
Lesson 2	Coordinate List representation stores each non-zero value as row, column, and value.
Lesson 3	A sparse matrix can be displayed as a full matrix using nested loops and a tracker variable.
Lesson 4	Sparse matrix addition uses a merge-style algorithm with <code>i</code> , <code>j</code> , and <code>k</code> .
Lesson 5	The <code>add()</code> function creates a result matrix and returns it as a pointer.
Lesson 6	The full C program uses <code>malloc</code> , <code>free</code> , and pointer-based functions.
Lesson 7	The sparse matrix program is converted into a C++ class.
Lesson 8	The C++ sparse matrix class is improved using operator overloading.

Lesson 8 is the final sparse matrix lesson before moving into polynomial data structures.

The main sparse matrix logic does **not** change in Lesson 8.

The big change is the way we write the code.

2. Main Idea of Lesson 8

In Lesson 7, we used normal member functions like this:

```
s1.read();  
s1.display();
```

In Lesson 8, we want to use more natural C++ syntax:

```
cin >> s1;  
cout << s1;  
sum = s1 + s2;
```

This is called **operator overloading**.

Operator overloading means:

Giving an existing C++ operator a special meaning for our own class.

For example, normally `+` adds numbers:

```
int c = a + b;
```

But in Lesson 8, we want `+` to add two sparse matrix objects:

```
SparseMatrix sum = s1 + s2;
```

So, for our `SparseMatrix` class:

Operator	New meaning
<code>>></code>	Read a sparse matrix object
<code><<</code>	Display a sparse matrix object
<code>+</code>	Add two sparse matrix objects

3. Why Operator Overloading Is Useful

Operator overloading makes the code easier to read.

Without operator overloading:

```
s1.read();  
s2.read();  
sum = s1.add(s2);  
s1.display();  
s2.display();  
sum.display();
```

With operator overloading:

```
cin >> s1;
cin >> s2;
sum = s1 + s2;
cout << s1;
cout << s2;
cout << sum;
```

The second version looks more like normal C++.

It is also easier to understand:

```
sum = s1 + s2;
```

This clearly means:

Add `s1` and `s2`, then store the result in `sum`.

4. Important Sparse Matrix Reminder

A sparse matrix is a matrix where most values are zero.

Example:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

This is a `5 x 5` matrix.

Total positions:

```
5 x 5 = 25
```

But only 5 values are non-zero.

So we store only these 5 values:

```
(0, 0, 1)
(1, 1, 1)
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

Each stored value contains:

```
row, column, value
```

This is Coordinate List representation.

5. Data Structure Used in Lesson 8

We still use the same `Element` structure from previous lessons.

```
struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};
```

Meaning of each field:

Field	Meaning
<code>i</code>	row index
<code>j</code>	column index
<code>x</code>	value stored at that position

Example:

```
Element e;
e.i = 2;
e.j = 3;
e.x = 7;
```

This means:

At row 2, column 3, the value is 7.

So the coordinate is:

(2, 3, 7)

6. SparseMatrix Class in Lesson 8

The sparse matrix is stored using a C++ class.

```
class SparseMatrix {
private:
    int m;
    int n;
    int num;
    Element *ele;

public:
    SparseMatrix(int m, int n, int num);
    ~SparseMatrix();

    friend istream& operator>>(istream& is, SparseMatrix& s);
    friend ostream& operator<<(ostream& os, SparseMatrix& s);

    SparseMatrix operator+(SparseMatrix& s);
};
```

The class contains:

1. Private data
2. Public functions
3. Friend operator functions
4. An overloaded `+` operator

7. Private Data Members

The private part stores the matrix information.

```
private:
    int m;
    int n;
    int num;
    Element *ele;
```

Meaning:

Member	Meaning
<code>m</code>	number of rows
<code>n</code>	number of columns
<code>num</code>	number of non-zero elements
<code>ele</code>	pointer to the dynamic array of non-zero elements

Important:

`num` does not mean $m \times n$.

`num` means only the number of non-zero values.

Example:

```
m = 5
n = 5
num = 5
```

This means:

The full matrix has 25 positions.
Only 5 values are non-zero.
Only 5 values are stored.

8. Why the Data Members Are Private

The data members are private because they should not be changed directly from outside the class.

This is not allowed:

```
s1.m = 5;
```

Why?

Because `m` is private.

Private data protects the object from being changed incorrectly.

This idea is called **encapsulation**.

Simple meaning of encapsulation:

Keep data and related functions together inside a class, and protect the data from direct outside access.

9. Constructor

A constructor runs automatically when an object is created.

In Lesson 8, the constructor can be written like this:

```
SparseMatrix(int m, int n, int num) {  
    this->m = m;  
    this->n = n;  
    this->num = num;  
    ele = new Element[this->num];  
}
```

When we write:

```
SparseMatrix s1(5, 5, 5);
```

The constructor runs automatically.

It stores:

```
m = 5  
n = 5  
num = 5
```

Then it allocates memory for 5 non-zero elements:

```
ele = new Element[this->num];
```

This creates:

```
ele[0]  
ele[1]  
ele[2]  
ele[3]  
ele[4]
```

10. The `this` Pointer

Inside the constructor, the parameter names are the same as the class member names:

```
SparseMatrix(int m, int n, int num)
```

The class also has members named:

```
int m;  
int n;  
int num;
```

So we use `this->` to avoid confusion.

```
this->m = m;  
this->n = n;  
this->num = num;
```

Meaning:

Code	Meaning
<code>this->m</code>	the object's <code>m</code>
<code>m</code>	the constructor parameter <code>m</code>

So:


```
this->m = m;
```

means:

Store the parameter `m` inside the object's `m` variable.

Simple meaning of `this`:

`this` refers to the current object.

11. Dynamic Memory Allocation Using `new[]`

The sparse matrix stores only non-zero elements.

So we allocate memory based on `num`, not based on `m * n`.

Correct:

```
ele = new Element[this->num];
```

Wrong:

```
ele = new Element[m * n];
```

Why is the wrong version bad?

Because it creates space for every matrix position, including zeros.

That defeats the purpose of sparse matrix storage.

12. Destructor

A destructor runs automatically when an object is destroyed.

In Lesson 8, the destructor releases the dynamic array:

```
~SparseMatrix() {  
    delete[] ele;  
}
```

The constructor creates memory:

```
ele = new Element[this->num];
```

The destructor releases memory:

```
delete[] ele;
```

This prevents memory leaks.

Important memory rule:

Allocation	Deallocation
<code>new</code>	<code>delete</code>
<code>new[]</code>	<code>delete[]</code>
<code>malloc</code>	<code>free</code>

Because `ele` was created using `new[]`, it must be released using `delete[]`.

13. Overloading the Input Operator >>

Normally, `>>` is used like this:

```
cin >> x;
```

In Lesson 8, we want:

```
cin >> s1;
```

This should read the non-zero elements of the sparse matrix.

So `cin >> s1` replaces:

```
s1.read();
```

The overloaded input operator reads:

```
row column value
```

for each non-zero element.

Example input:

```
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

This stores:

```
(0, 0, 1)
(1, 1, 1)
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

14. Input Operator Function Header

The input operator is declared like this inside the class:

```
friend istream& operator>>(istream& is, SparseMatrix& s);
```

Meaning of each part:

Part	Meaning
<code>friend</code>	allows the function to access private members
<code>istream&</code>	returns the input stream
<code>operator>></code>	overloads the <code>>></code> operator

Part	Meaning
<code>istream& is</code>	input stream, usually <code>cin</code>
<code>SparseMatrix& s</code>	sparse matrix object to fill

The full function is written outside the class:

```
istream& operator>>(istream& is, SparseMatrix& s) {
    cout << "Enter non-zero elements as row column value:\n";

    for (int i = 0; i < s.num; i++) {
        is >> s.ele[i].i >> s.ele[i].j >> s.ele[i].x;
    }

    return is;
}
```

15. Why `operator>>` Uses `SparseMatrix& s`

The parameter is:

```
SparseMatrix& s
```

This means `s` is passed by reference.

We pass by reference because the function must modify the original object.

If we passed by value, the function would receive a copy.

Then the original sparse matrix would not be filled properly.

So this is correct:

```
istream& operator>>(istream& is, SparseMatrix& s)
```

16. Why `operator>>` Returns `istream&`

The function ends with:

```
return is;
```

This allows chaining.

Example:

```
cin >> s1 >> s2;
```

This works like:

```
cin >> s1;  
cin >> s2;
```

If we did not return `is`, chaining would not work.

Wrong:

```
void operator>>(istream& is, SparseMatrix& s)
```

Correct:

```
istream& operator>>(istream& is, SparseMatrix& s)
```

17. Overloading the Output Operator <<

Normally, << is used like this:

```
cout << x;
```

In Lesson 8, we want:

```
cout << s1;
```

This should display the full sparse matrix.

So `cout << s1` replaces:

```
s1.display();
```

The matrix is stored only as non-zero elements, but `operator<<` prints the full grid.

Example stored form:

```
(0, 0, 1)
(1, 1, 1)
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

Output:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

18. Output Operator Function Header

The output operator is declared inside the class like this:

```
friend ostream& operator<<(ostream& os, SparseMatrix& s);
```

Meaning of each part:

Part	Meaning
<code>friend</code>	allows access to private members
<code>ostream&</code>	returns the output stream
<code>operator<<</code>	overloads the <code><<</code> operator
<code>ostream& os</code>	output stream, usually <code>cout</code>
<code>SparseMatrix& s</code>	sparse matrix object to display

The full function is written outside the class:

```
ostream& operator<<(ostream& os, SparseMatrix& s) {
    int k = 0;

    for (int i = 0; i < s.m; i++) {
        for (int j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                os << s.ele[k].x << " ";
                k++;
            } else {
                os << "0 ";
            }
        }
        os << endl;
    }

    return os;
}
```

19. Why `operator<<` Returns `ostream&`

The function ends with:

```
return os;
```

This allows output chaining.

Example:

```
cout << s1 << s2;
```

This works like:

```
cout << s1;
cout << s2;
```

If we did not return `os`, chaining would not work.

20. Why `>>` and `<<` Are Friend Functions

The class data is private:

```
private:
    int m;
    int n;
    int num;
    Element *ele;
```

But `operator>>` needs to access:

```
s.num
s.ele
```

And `operator<<` needs to access:

```
s.m
s.n
s.num
s.ele
```

Normally, outside functions cannot access private data.

So we declare them as `friend`.

```
friend istream& operator>>(istream& is, SparseMatrix& s);
friend ostream& operator<<(ostream& os, SparseMatrix& s);
```

A friend function is not a normal member function, but it is allowed to access private data.

21. Another Reason `>>` and `<<` Are Usually Friends

Look at this code:

```
cin >> s1;
```

The left side is `cin`, not `s1`.

Similarly:

```
cout << s1;
```

The left side is `cout`, not `s1`.

Because the left operand is a stream object, `>>` and `<<` are usually written as non-member friend functions.

This makes the syntax natural:

```
cin >> s1;  
cout << s1;
```

22. How `operator<<` Displays the Matrix

The display logic uses nested loops.

```
for (int i = 0; i < s.m; i++) {  
    for (int j = 0; j < s.n; j++) {  
        // check current position  
    }  
}
```

The outer loop moves row by row.

The inner loop moves column by column.

So the program visits every visible matrix position.

For each position `(i, j)`, it checks whether the current stored non-zero element matches that position.

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

If true:

```
os << s.ele[k].x << " ";  
k++;
```

If false:

```
os << "0 ";
```

23. The Tracker Variable `k`

In `operator<<`, the variable `k` tracks the current non-zero element.

```
int k = 0;
```

At the beginning:

```
k = 0
```

So the current stored element is:

```
s.ele[0]
```

After printing that element, we do:

```
k++;
```

Then `k` points to the next stored element.

Beginner mental model:

`k` is like a finger pointing to the next non-zero element that may need to be printed.

24. Why We Check `k < s.num`

The display condition is:

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

The first part is:

```
k < s.num
```

This prevents the program from reading outside the array.

Example:

If:

```
num = 5
```

Valid indexes are:

```
0, 1, 2, 3, 4
```

When `k` becomes `5`, this is invalid:

```
s.ele[5]
```

So we check:

```
k < s.num
```

before accessing:

```
s.ele[k]
```

25. Row-Major Order Assumption

The display and addition algorithms assume the non-zero elements are stored in row-major order.

Row-major order means:

1. Sort by row first.
2. If rows are equal, sort by column.

Good order:

```
0 0 1
0 3 5
1 2 7
3 1 4
```

Bad order:

```
3 1 4
0 0 1
1 2 7
0 3 5
```

Why is bad order a problem?

Because the algorithm moves forward only.

It does not go backward and search again.

So if the stored elements are not sorted, the output or addition result may be wrong.

26. Overloading the Addition Operator

Now we want this:

```
SparseMatrix sum = s1 + s2;
```

This should add two sparse matrices.

The overloaded  operator replaces the older addition function idea.

Instead of writing something like:

```
sum = add(s1, s2);
```

we write:

```
sum = s1 + s2;
```

This is cleaner and easier to read.

27. `operator+` as a Member Function

The addition operator is written as a member function:

```
SparseMatrix operator+(SparseMatrix& s)
```

When we write:

```
sum = s1 + s2;
```

C++ understands it like this:

```
sum = s1.operator+(s2);
```

So inside `operator+` :

Part	Meaning
current object	<code>s1</code>
parameter <code>s</code>	<code>s2</code>
returned value	result sparse matrix

Inside the function:

```
m  
n  
num  
ele
```

belong to the current object, which is `s1`.

And:

```
s.m  
s.n  
s.num  
s.ele
```

belong to the second object, which is `s2`.

28. First Step in `operator+`: Dimension Check

Two matrices can be added only if they have the same number of rows and columns.

So we check:

```
if (m != s.m || n != s.n) {  
    return SparseMatrix(0, 0, 0);  
}
```

This means:

If row counts are different, addition is impossible.
If column counts are different, addition is impossible.

Example allowed:

$5 \times 5 + 5 \times 5$

Example not allowed:

$5 \times 5 + 4 \times 5$

Matrix addition works position by position.

So every position in one matrix must match a position in the other matrix.

29. Result Matrix Size

When adding sparse matrices, we do not know the exact result size at first.

So we allocate enough space for the worst case.

Worst case:

No coordinates overlap.

If:

```
s1 has 5 non-zero elements  
s2 has 5 non-zero elements
```

The result may need:

```
5 + 5 = 10 elements
```

So we create:

```
SparseMatrix sum(m, n, num + s.num);
```

This gives the result enough space.

The actual number of result elements may be smaller.

That is why we later set:

```
sum.num = k;
```

30. Why the Result May Be Smaller

If both matrices have a non-zero value at the same coordinate, those values are added and stored once.

Example:

```
s1 has (2, 5, 7)  
s2 has (2, 5, 6)
```

Both are at:

```
row = 2, column = 5
```

So we add:

```
7 + 6 = 13
```

The result stores only:

```
(2, 5, 13)
```

Two input elements became one result element.

So the final result count may be smaller than:

```
num + s.num
```

31. The Three Pointers: `i`, `j`, and `k`

Sparse matrix addition uses three index variables.

```
int i = 0;
int j = 0;
int k = 0;
```

Meaning:

Variable	Meaning
<code>i</code>	current element in the first matrix/current object
<code>j</code>	current element in the second matrix/parameter object <code>s</code>
<code>k</code>	current writing position in the result matrix <code>sum</code>

Beginner mental model:

Think of `i`, `j`, and `k` as three fingers.

- `i` points to the current element in `s1`.
- `j` points to the current element in `s2`.
- `k` points to where the next result element should be written.

32. Main Loop of Addition

The main loop is:

```
while (i < num && j < s.num) {  
    // compare elements  
}
```

This means:

Keep comparing while both sparse matrices still have elements left.

The loop stops when one matrix finishes.

After that, leftover elements from the other matrix are copied.

33. Addition Rule 1: Current Object Row Is Smaller

Code:

```
if (ele[i].i < s.ele[j].i) {  
    sum.ele[k++] = ele[i++];  
}
```

Meaning:

The current element in the first matrix comes before the current element in the second matrix.

So copy the first matrix element into the result.

Then move:

```
i forward  
k forward
```

Example:

```
s1 element = (1, 3, 7)  
s2 element = (2, 1, 5)
```

Compare rows:

1 < 2

So copy:

(1, 3, 7)

34. Addition Rule 2: Parameter Object Row Is Smaller

Code:

```
else if (ele[i].i > s.ele[j].i) {  
    sum.ele[k++] = s.ele[j++];  
}
```

Meaning:

The current element in the second matrix comes before the current element in the first matrix.

So copy the second matrix element into the result.

Then move:

j forward
k forward

Example:

```
s1 element = (3, 0, 4)  
s2 element = (1, 2, 8)
```

Compare rows:

3 > 1

So copy:

(1, 2, 8)

35. Addition Rule 3: Rows Equal, First Column Is Smaller

If the rows are equal, compare columns.

Code:

```
if (ele[i].j < s.ele[j].j) {  
    sum.ele[k++] = ele[i++];  
}
```

Example:

```
s1 element = (2, 1, 6)  
s2 element = (2, 4, 9)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
1 < 4
```

So copy from the first matrix:

```
(2, 1, 6)
```

Then move:

```
i forward  
k forward
```

36. Addition Rule 4: Rows Equal, Second Column Is Smaller

Code:

```
else if (ele[i].j > s.ele[j].j) {  
    sum.ele[k++] = s.ele[j++];  
}
```

Example:

```
s1 element = (2, 5, 6)  
s2 element = (2, 3, 9)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
5 > 3
```

So copy from the second matrix:

```
(2, 3, 9)
```

Then move:

```
j forward  
k forward
```

37. Addition Rule 5: Same Row and Same Column

This is the real addition case.

Code:

```
else {  
    sum.ele[k].i = ele[i].i;  
    sum.ele[k].j = ele[i].j;  
    sum.ele[k].x = ele[i].x + s.ele[j].x;  
    i++;  
    j++;  
    k++;  
}
```

Example:

```
s1 element = (2, 5, 7)  
s2 element = (2, 5, 6)
```

Same row:

```
2 == 2
```

Same column:

```
5 == 5
```

So add the values:

```
7 + 6 = 13
```

Store:

```
(2, 5, 13)
```

Then move:

```
i++  
j++  
k++
```

Why move both `i` and `j`?

Because one element from each matrix was used.

Why move `k`?

Because one result element was written.

38. Copying Leftovers

The main loop runs only while both matrices still have elements:

```
while (i < num && j < s.num)
```

If one matrix finishes first, the other matrix may still have remaining elements.

These are called leftovers.

They must be copied into the result.

Copy leftovers from the first matrix:

```
while (i < num) {
    sum.ele[k++] = ele[i++];
}
```

Copy leftovers from the second matrix:

```
while (j < s.num) {
    sum.ele[k++] = s.ele[j++];
}
```

Why copy leftovers?

Because if a value exists only in one matrix, it remains unchanged in the sum.

39. Why We Set `sum.num = k`

At first, the result matrix was created with maximum possible capacity:

```
num + s.num
```

But this is only the maximum possible size.

The actual number of result elements is counted by `k`.

Every time we write one result element, we do:

```
k++;
```

So at the end:

```
sum.num = k;
```

Wrong:

```
sum.num = num + s.num;
```

Correct:

```
sum.num = k;
```

Why?

Because overlapping coordinates reduce the final number of stored elements.

40. Full `operator+` Function

```
SparseMatrix operator+(SparseMatrix& s) {
    if (m != s.m || n != s.n) {
        return SparseMatrix(0, 0, 0);
    }

    SparseMatrix sum(m, n, num + s.num);

    int i = 0;
    int j = 0;
    int k = 0;

    while (i < num && j < s.num) {
        if (ele[i].i < s.ele[j].i) {
            sum.ele[k++] = ele[i++];
        }
        else if (ele[i].i > s.ele[j].i) {
```

```

        sum.ele[k++] = s.ele[j++];
    }
    else {
        if (ele[i].j < s.ele[j].j) {
            sum.ele[k++] = ele[i++];
        }
        else if (ele[i].j > s.ele[j].j) {
            sum.ele[k++] = s.ele[j++];
        }
        else {
            sum.ele[k].i = ele[i].i;
            sum.ele[k].j = ele[i].j;
            sum.ele[k].x = ele[i].x + s.ele[j].x;
            i++;
            j++;
            k++;
        }
    }
}

while (i < num) {
    sum.ele[k++] = ele[i++];
}

while (j < s.num) {
    sum.ele[k++] = s.ele[j++];
}

sum.num = k;
return sum;
}

```

41. Complete Lesson 8 Program

```

#include <iostream>
using namespace std;

struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};

class SparseMatrix {

```



```

private:
    int m;
    int n;
    int num;
    Element *ele;

public:
    SparseMatrix(int m, int n, int num) {
        this->m = m;
        this->n = n;
        this->num = num;
        ele = new Element[this->num];
    }

    ~SparseMatrix() {
        delete[] ele;
    }

    friend istream& operator>>(istream& is, SparseMatrix& s);
    friend ostream& operator<<(ostream& os, SparseMatrix& s);

    SparseMatrix operator+(SparseMatrix& s) {
        if (m != s.m || n != s.n) {
            return SparseMatrix(0, 0, 0);
        }

        SparseMatrix sum(m, n, num + s.num);

        int i = 0;
        int j = 0;
        int k = 0;

        while (i < num && j < s.num) {
            if (ele[i].i < s.ele[j].i) {
                sum.ele[k++] = ele[i++];
            }
            else if (ele[i].i > s.ele[j].i) {
                sum.ele[k++] = s.ele[j++];
            }
            else {
                if (ele[i].j < s.ele[j].j) {
                    sum.ele[k++] = ele[i++];
                }
                else if (ele[i].j > s.ele[j].j) {
                    sum.ele[k++] = s.ele[j++];
                }
                else {
                    sum.ele[k].i = ele[i].i;

```

```

        sum.ele[k].j = ele[i].j;
        sum.ele[k].x = ele[i].x + s.ele[j].x;
        i++;
        j++;
        k++;
    }
}

while (i < num) {
    sum.ele[k++] = ele[i++];
}

while (j < s.num) {
    sum.ele[k++] = s.ele[j++];
}

sum.num = k;
return sum;
}

};

istream& operator>>(istream& is, SparseMatrix& s) {
    cout << "Enter non-zero elements as row column value:\n";

    for (int i = 0; i < s.num; i++) {
        is >> s.ele[i].i >> s.ele[i].j >> s.ele[i].x;
    }

    return is;
}

ostream& operator<<(ostream& os, SparseMatrix& s) {
    int k = 0;

    for (int i = 0; i < s.m; i++) {
        for (int j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                os << s.ele[k].x << " ";
                k++;
            } else {
                os << "0 ";
            }
        }
        os << endl;
    }

    return os;
}

```

```

}

int main() {
    SparseMatrix s1(5, 5, 5);
    SparseMatrix s2(5, 5, 5);

    cout << "Enter first sparse matrix:\n";
    cin >> s1;

    cout << "Enter second sparse matrix:\n";
    cin >> s2;

    SparseMatrix sum = s1 + s2;

    cout << "\nFirst Matrix:\n";
    cout << s1;

    cout << "\nSecond Matrix:\n";
    cout << s2;

    cout << "\nSum Matrix:\n";
    cout << sum;

    return 0;
}

```

42. Explanation of `main()`

```

SparseMatrix s1(5, 5, 5);
SparseMatrix s2(5, 5, 5);

```

This creates two sparse matrix objects.

Each matrix has:

```

5 rows
5 columns
5 non-zero elements

```

Then:

```
cin >> s1;  
cin >> s2;
```

This reads the non-zero elements of both matrices.

Then:

```
SparseMatrix sum = s1 + s2;
```

This calls the overloaded `+` operator.

Then:

```
cout << s1;  
cout << s2;  
cout << sum;
```

This displays the first matrix, second matrix, and sum matrix.

43. Dry Run Example

Let the first matrix `s1` be a diagonal matrix of `1` s.

Input for `s1`:

```
0 0 1  
1 1 1  
2 2 1  
3 3 1  
4 4 1
```

Stored form:

```
s1.ele[0] = (0, 0, 1)  
s1.ele[1] = (1, 1, 1)  
s1.ele[2] = (2, 2, 1)  
s1.ele[3] = (3, 3, 1)  
s1.ele[4] = (4, 4, 1)
```

Full matrix view:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

Let the second matrix `s2` be first-row `5` s.

Input for `s2`:

```
0 0 5
0 1 5
0 2 5
0 3 5
0 4 5
```

Stored form:

```
s2.ele[0] = (0, 0, 5)
s2.ele[1] = (0, 1, 5)
s2.ele[2] = (0, 2, 5)
s2.ele[3] = (0, 3, 5)
s2.ele[4] = (0, 4, 5)
```

Full matrix view:

```
5 5 5 5 5
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Now we calculate:

```
sum = s1 + s2;
```

44. Dry Run Step 1: First Comparison

Initial values:

```
i = 0  
j = 0  
k = 0
```

Current elements:

```
s1.ele[i] = s1.ele[0] = (0, 0, 1)  
s2.ele[j] = s2.ele[0] = (0, 0, 5)
```

Compare rows:

```
0 == 0
```

Rows are equal, so compare columns:

```
0 == 0
```

Same row and same column.

So add values:

```
1 + 5 = 6
```

Store:

```
sum.ele[0] = (0, 0, 6)
```

Move:

```
i = 1  
j = 1  
k = 1
```

45. Dry Run Step 2: Compare Next Elements

Current elements:

```
s1.ele[1] = (1, 1, 1)
s2.ele[1] = (0, 1, 5)
```

Compare rows:

```
1 > 0
```

So the `s2` element comes first.

Copy:

```
sum.ele[1] = (0, 1, 5)
```

Move:

```
j = 2
k = 2
```

`i` stays the same because the `s1` element was not used yet.

46. Dry Run Step 3: Continue Copying First-Row Values

Now the same idea continues.

Copy remaining first-row values from `s2`:

```
sum.ele[2] = (0, 2, 5)
sum.ele[3] = (0, 3, 5)
sum.ele[4] = (0, 4, 5)
```

Now `s2` is finished.

So:

```
j = 5
```

The main comparison loop stops because `j == s2.num`.

47. Dry Run Step 4: Copy Leftovers from `s1`

`s1` still has remaining diagonal elements:

```
(1, 1, 1)
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

Copy them into the result:

```
sum.ele[5] = (1, 1, 1)
sum.ele[6] = (2, 2, 1)
sum.ele[7] = (3, 3, 1)
sum.ele[8] = (4, 4, 1)
```

Final value:

```
k = 9
```

So:

```
sum.num = 9;
```

48. Final Dry Run Result

Final stored form:

```
(0, 0, 6)
(0, 1, 5)
(0, 2, 5)
(0, 3, 5)
(0, 4, 5)
```



```
(1, 1, 1)
(2, 2, 1)
(3, 3, 1)
(4, 4, 1)
```

Full matrix output:

```
6 5 5 5 5
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

Why is `(0, 0)` equal to `6`?

Because:

```
1 + 5 = 6
```

Why are the other first-row values copied?

Because they exist only in `s2`.

Why are the other diagonal values copied?

Because they exist only in `s1`.

49. Complexity Analysis

`operator>>` **Time Complexity**

The input operator reads each non-zero element once.

If there are `num` non-zero elements:

```
Time complexity = O(num)
```

`operator<<` **Time Complexity**

The output operator displays the full matrix.

If the matrix has `m` rows and `n` columns:

Total visible positions = $m \times n$

So:

Time complexity = $O(m \times n)$

Even though sparse storage saves memory, displaying the full matrix still visits every visible position.

`operator+` Time Complexity

The addition operator scans each stored non-zero element at most once.

If the first matrix has `num` non-zero elements and the second matrix has `s.num` non-zero elements:

Time complexity = $O(\text{num} + \text{s.num})$

`operator+` Space Complexity

The result may need space for all elements from both matrices.

Worst case:

No coordinates overlap.

So:

Space complexity = $O(\text{num} + \text{s.num})$

50. Complexity Table

Operation	Meaning	Time Complexity	Extra/Result Space
<code>operator>></code>	Reads non-zero elements	$O(\text{num})$	$O(1)$ extra
<code>operator<<</code>	Displays full matrix	$O(m \times n)$	$O(1)$ extra
<code>operator+</code>	Adds two sparse matrices	$O(\text{num} + \text{s.num})$	$O(\text{num} + \text{s.num})$

Operation	Meaning	Time Complexity	Extra/Result Space
Sparse storage	Stores only non-zero elements	—	<code>O(num)</code>
Full matrix storage	Stores all positions	—	<code>O(m x n)</code>

51. Important Professional C++ Note: Raw Pointer Copying Issue

The lesson code is useful for learning operator overloading, but there is an important C++ warning.

The class uses a raw pointer:

```
Element *ele;
```

It also has a destructor:

```
~SparseMatrix() {
    delete[] ele;
}
```

When a class has a raw pointer and a destructor, copying can become dangerous if copy behavior is not handled correctly.

This is related to the **Rule of Three**.

The Rule of Three says that if a class needs a destructor, it usually also needs:

1. A copy constructor
2. A copy assignment operator
3. A destructor

Why?

Because returning objects by value can create copies.

Example:

```
return sum;
```

A professional C++ version would usually use:

```
std::vector<Element>
```

instead of raw dynamic arrays, or it would define proper copy behavior.

For this lesson, focus on understanding operator overloading and the sparse matrix algorithm.

52. Common Beginner Mistakes

Mistake 1: Forgetting `friend`

Wrong idea:

```
istream& operator>>(istream& is, SparseMatrix& s)
```

without declaring it as a friend inside the class.

Problem:

The function cannot access private data like:

```
s.ele  
s.num
```

Correct:

```
friend istream& operator>>(istream& is, SparseMatrix& s);
```

Mistake 2: Not Returning the Stream

Wrong:

```
void operator>>(istream& is, SparseMatrix& s)
```

Correct:

```
istream& operator>>(istream& is, SparseMatrix& s)
```

Why?

Because returning the stream allows chaining:

```
cin >> s1 >> s2;
```

Mistake 3: Passing the Matrix by Value in `operator>>`

Wrong:

```
istream& operator>>(istream& is, SparseMatrix s)
```

Correct:

```
istream& operator>>(istream& is, SparseMatrix& s)
```

Why?

The input operator must modify the original object.

Passing by value would modify only a copy.

Mistake 4: Comparing Columns Before Rows

Wrong idea:

```
Compare columns first.
```

Correct idea:

```
Compare rows first.  
If rows are equal, compare columns.
```

Why?

The sparse elements are stored in row-major order.

Mistake 5: Moving Only One Pointer When Coordinates Match

Wrong:

```
same coordinate:  
add values  
i++  
k++
```

Correct:

```
same coordinate:  
add values  
i++  
j++  
k++
```

Why?

One element from the first matrix and one element from the second matrix were both used.

Mistake 6: Forgetting to Copy Leftovers

Wrong idea:

```
Stop when one matrix finishes.
```

Correct idea:

```
After one matrix finishes, copy all remaining elements from the other matrix.
```

Mistake 7: Setting Result Count Incorrectly

Wrong:

```
sum.num = num + s.num;
```

Correct:

```
sum.num = k;
```

Why?

`k` stores the actual number of result elements written.

Mistake 8: Forgetting Row-Major Order

Wrong idea:

```
Elements can be entered randomly.
```

Correct idea:

```
Elements should be sorted by row first and column second.
```

Why?

Both display and addition move forward only.

They do not search the whole array repeatedly.

Mistake 9: Allocating `m * n` Elements

Wrong:

```
ele = new Element[m * n];
```

Correct:

```
ele = new Element[num];
```

Why?

A sparse matrix should store only non-zero elements.

Mistake 10: Forgetting `delete[]`

Wrong:

```
delete ele;
```

Correct:

```
delete[] ele;
```

Why?

The memory was created using:

```
new Element[num]
```

So it must be released using:

```
delete[] ele;
```

53. Beginner Mental Model

Think of a sparse matrix object as a smart box.

Inside the box, it stores:

```
m  
n  
num  
ele
```

In Lesson 7, the box had normal functions:

```
s1.read();  
s1.display();
```

In Lesson 8, we teach the box to understand normal C++ symbols:


```
cin >> s1;
cout << s1;
s1 + s2;
```

So operator overloading gives the object a more natural language.

The object still stores data the same way.

The object still adds using the same merge algorithm.

Only the syntax becomes cleaner.

54. One-Line Memory Trick

Operator overloading lets sparse matrix objects use natural C++ syntax like `cin >> s1`, `cout << s1`, and `s1 + s2`, while the internal logic stays the same.

55. Important Code Symbols

Symbol	Meaning
<code>operator>></code>	overloaded input operator
<code>operator<<</code>	overloaded output operator
<code>operator+</code>	overloaded addition operator
<code>friend</code>	allows a non-member function to access private class data
<code>istream</code>	input stream type, used with <code>cin</code>
<code>ostream</code>	output stream type, used with <code>cout</code>
<code>return is</code>	returns input stream for chaining
<code>return os</code>	returns output stream for chaining
<code>SparseMatrix& s</code>	reference to a sparse matrix object
<code>this-></code>	refers to the current object's member
<code>new[]</code>	creates a dynamic array
<code>delete[]</code>	releases a dynamic array
<code>i</code>	current element in first matrix

Symbol	Meaning
j	current element in second matrix
k	current position in result matrix or display tracker
num	number of non-zero elements
m	number of rows
n	number of columns
ele	array of non-zero elements

56. Lesson 8 Summary

Lesson 8 improves the C++ sparse matrix class by using operator overloading.

The three main overloaded operators are:

```
cin >> s1;
cout << s1;
sum = s1 + s2;
```

`operator>>` reads non-zero elements.

`operator<<` displays the full matrix.

`operator+` adds two sparse matrices using the same merge algorithm from previous lessons.

`>>` and `<<` are declared as friend functions because they need to access private class data.

Both stream operators return the stream object so chaining works.

The addition operator uses:

```
i, j, k
```

to merge two sorted Coordinate Lists.

The result array is first created with maximum possible size:

```
num + s.num
```

But the final count is:

```
sum.num = k;
```

because overlapping coordinates reduce the actual number of stored result elements.

The most important idea is:

Lesson 8 does not change the sparse matrix algorithm. It gives the sparse matrix class cleaner C++ syntax.

57. Quick Self-Check Questions

1. What does operator overloading mean?
2. What does `cin >> s1` replace from Lesson 7?
3. What does `cout << s1` replace from Lesson 7?
4. Why are `operator>>` and `operator<<` declared as `friend`?
5. Why should `operator>>` return `istream&`?
6. Why should `operator<<` return `ostream&`?
7. In `s1 + s2`, which object is the current object?
8. In `s1 + s2`, what does the parameter `s` represent inside `operator+`?
9. Why must matrix dimensions match before addition?
10. Why is the result array allocated with size `num + s.num`?
11. Why do we set `sum.num = k` at the end?
12. Why must sparse elements be stored in row-major order?
13. What is the time complexity of `operator>>`?
14. What is the time complexity of `operator<<`?
15. What is the time complexity of `operator+`?

58. Final Takeaway

By the end of Lesson 8, you should be able to read this code clearly:

```
cin >> s1;  
cin >> s2;  
SparseMatrix sum = s1 + s2;  
cout << sum;
```

And understand that it means:

1. Read the first sparse matrix.
2. Read the second sparse matrix.
3. Add them using sparse matrix merge logic.
4. Display the result as a full matrix.

This completes the sparse matrix part of the course and prepares you for the next unit: polynomial data structures.